

Assignment 2 Submission

Nopparuj Sodsri (A0305649X)

CS3210 – 2025/26 Semester 2

1 Implementation

The implementation has 3 stages. It first allocates memory of viral sequences ("**signature**") and array of confidences into GPU memory, followed by loading each patient's DNA sequence ("**sample**") and quality hash into GPU and performs matching against each virus signature sample by sample before collecting confidence and integrity and freeing memory from the GPU. Lastly, the program frees all memory of viruses and confidence lists.

1.1 Memory Allocation of Sample and Signature

Since the cost of CUDA API call like `cudaMemcpy` is expensive, we concatenate all viral sequences into one contiguous string (each sequence is actually padded with garbage character to align 16-byte window for optimization later explained in 3.1).

For each matcher kernel, one sample's sequence and quality and flattened string of signatures are passed into the matching kernel. The sample strings are immediately loaded from global into shared memory to reduce global access, and one signature is extracted from the flattened array per iteration.

1.2 Sample-Signature Matching

Starting from window located at i -index ($i = 0, 1, \dots, l_{\text{samp}} - l_{\text{sig}}$), compare the character at index $i + j$ of sample with j -th character of signature for $j = 0, 1, \dots, l_{\text{sig}} - 1$, where l_{sig} is the length of a virus signature. It is possible to terminate screening and start with window of index $i + 1$ if any mismatch is found before reaching the end of window. However, the sample is longer than 1000000 characters, so parallelization is essential for optimal performance.

The parallelization Strategy is as follows. A sequence is divided and among blocks with of choice n_{subsamp} elements each. Each block scans through subsequence independently, where its i -th thread compare viral sequence with sample sequence's window at index of `threadIdx.x + i * blockDim.x` of the substring, for i starting with 0 and increasing by 1 until can't be fit within a window, as shown in Figure ???. Adjacent sub-sequences overlap by up to $\max l_{\text{sig}}$ characters. This overlap guarantees that every possible window in the original sample is fully covered by at least one block, avoiding false negatives at sub-sequence boundaries.

To minimise warp divergence from branching, character comparisons exploit the XOR bitmask approach. Two characters are XOR'ed, and the result is right-shifted then masked against 0xA80054. A non-zero result indicates a mismatch. This mask is constructed such that the bits at positions 2, 4, 6, 19, 21, and 23 are set—exactly the positions produced by XOR-ing mismatched DNA bases, as shown in Table 1

1.3 Best Match Determination and Synchronisation

Each thread maintains a local score variable that is updated as it steps through the window characters. If a mismatch is detected, the thread resets its localScore to -1 (marking the window as invalid) and stalls until the remaining threads in its warp either complete their window or also terminate early. At the end of a valid window (`localScore != -1`), the thread attempts an atomic update

^	A	T	C	G	N
A	0	2	6	21	15
T	2	0	4	23	13
C	6	4	0	19	9
G	8	23	19	0	26
N	15	13	9	26	0

Table 1: XOR lookup table for DNA bases

to the global `bestMatch` variable. No explicit intra-block synchronisation is required because each warp operates independently, and signatures are processed sequentially.

1.4 Confidence and Integrity Calculation

After confidence is updated by `kernelMatch`, a program copy confidence from GPU address into host function `hostMatch` and check if it is not -1, that is there is at least one valid match. If so, and another kernel `setIntegrity` is launched to compute the integrity score.

Unlike substring distribution in previous step, a hash quality string is partitioned among in contiguous manner, and the number of block is simply $\left\lceil \frac{l_{\text{samp}}}{\text{BLOCK_SIZE}} \right\rceil$. Each block accumulates its partial sum using efficient parallel reduction with warp unrolling (based on NVIDIA parallel reduction guide: <https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>), then adds its contribution to a global accumulator, which is taken modulo 97 to yield the final integrity value.

1.5 CUDA Streaming and Global Memory Allocation

To further accelerate parallelization of the program, we use CUDA stream to bypass serialization of GPU operation. A matcher kernel with i -th sample is run on stream $i \% \text{NUM_STREAM}$. The memory allocation for viral signatures is performed using a synchronous `cudaMalloc` call, since the flattened signature string is shared across all streams and must be fully resident before any kernel executes.

The benefit of streaming diminishes as the number of streams or sub-sequence sizes increases, because individual kernels already saturate the available Streaming Multiprocessors (SMs). This interaction between stream count and SM occupancy is analysed in Section 2.2.

2 Performance Analysis

2.1 Number of Threads per block

One of the most common factor of the performance of GPU programming is the number of threads in a block. Theoretically, higher number of threads leads to higher occupancy, but in practical situation, local variable might not fit in the register if there are too many threads, forcing them to use slow local memory and thereby degrade the performance.

The performance benchmarking results are shown in Figure 1. Execution time of both GPUs decrease the from 128 to 256, which is reasonable since 128 threads per block has very low theoretical occupancy. At beyond 512 threads, the performance plateau and can reverse slightly on some configurations. This could be attributed register pressure starting to offset occupancy benefits. Based on these empirical results, 512 threads per block was selected as the default for all subsequent experiments.

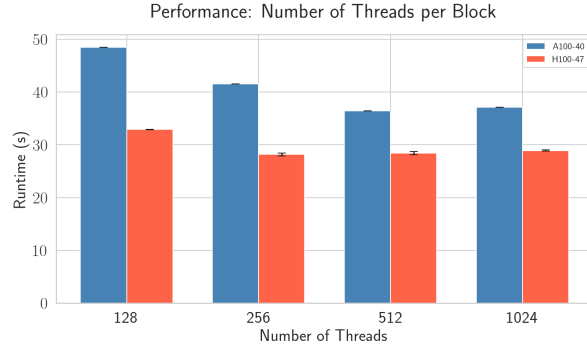


Figure 1: GPU Performance Scaling on Number of Threads Per Block

2.2 Performance Based on Number of Stream and Size of Sample

Although multiple streams enable us to concurrently launch kernels to optimize GPU utilization, as the number of streams or size of subsample is increased, the effect of streaming becomes less significant due to starvation of SMs. These two factors are therefore direct joint determinants of the speedup.

According to Figure 2, The trend of A100-40 and H100-47 are notably different. H100-47 converges to peak performance at just 2 streams and shows no further improvement beyond that point, while A100-40 continuously benefits from additional streams up to 16, albeit with diminishing returns.

A plausible explanation for this divergence lies in the architectural differences between the two GPUs. The H100 features a significantly higher per-SM throughput, meaning that even a few matching kernel is sufficient to saturate most of the H100's compute resources. As a result, additional streams cannot find idle SMs to occupy and provide no meaningful overlap. On the A100, which has lower per-SM throughput, there are some residual SMs that additional streams can occupy, which could explain the continued improvement. In both cases, the eventual plateau confirms that SM saturation is the dominant bottleneck.

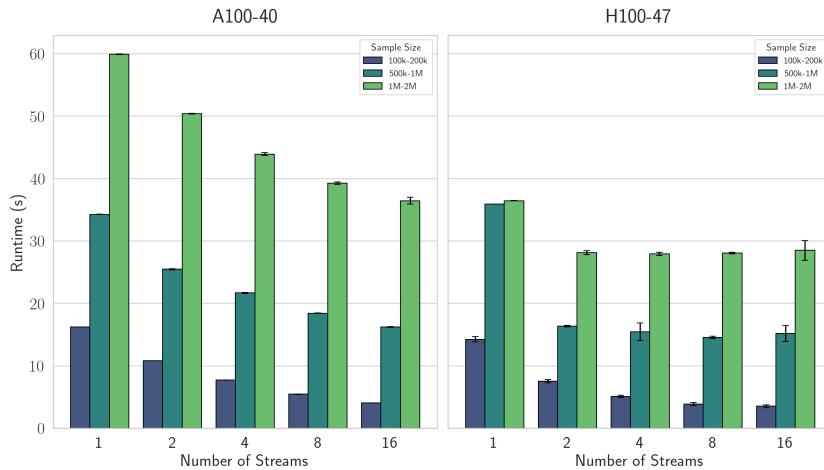


Figure 2: Performance Scaling of A100-40 (left) and H100-47 (right) between the number of streams and execution time for different lengths of sample sizes

2.3 Size of sample subsequence per Shared memory

The sub-sequence size per block controls a trade-off: larger sub-sequences reduce the number of blocks launched per kernel, but increase each block's shared memory footprint, limiting how many blocks can reside concurrently within a single SM.

In more details, a sample with length n_{sample} and maximum length of signature $\max l_{\text{sig}}$, the number of block, the number of block required for one kernel performing matching one sample is

$$\frac{\left\lceil \frac{l_{\text{sample}} + 2 \cdot N_{\text{sub}} - \max l_{\text{sig}} - 1}{2 \cdot N_{\text{sub}} - \max l_{\text{sig}}} \right\rceil}{\left\lfloor \frac{233472}{\max l_{\text{sig}} + 2 \cdot N_{\text{sub}}} \right\rfloor}.$$

The plot of this equation is shown in Figure 3a.

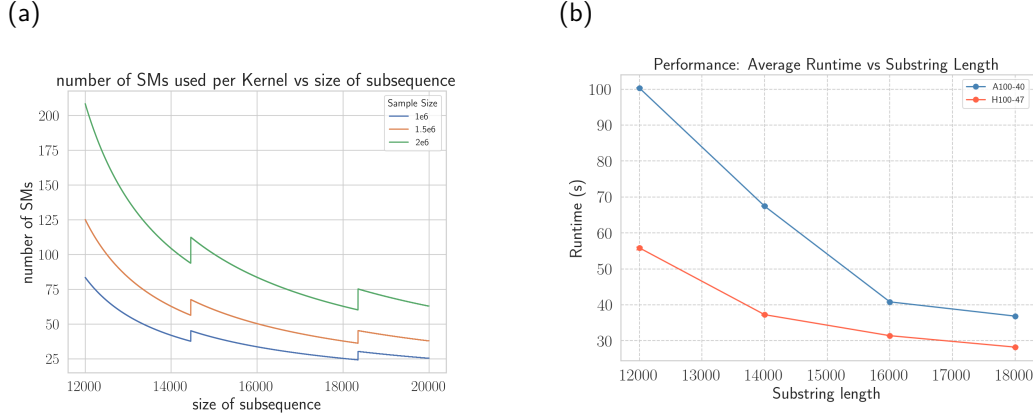


Figure 3: Analysis number of Streaming Multiprocessors (SM) being Called upon One Matching Kernel. (a) number of SMs as a function per kernel as a function of $N_{\text{subsequence}}$ with $\max l_{\text{sig}} = 10000$. (b) Execution scaling with the size of sample's substring size

The numerator is number of block to be called in a kernel, and the denominator is the number of block that can be fit within one SM. The simple explanation is that although higher size of subsequence for each block help achieve lower block needed for one kernel, it might limit the number of block that can fit inside one SM due to its limitation on shared memory. Note that 233472 bytes is total shared memory in H100-47's single SM, but the same logic can be applied for A100-40 with 167936 bytes of shared memory.

The result shown in Figure 3b is in agreement with the hypothesis, where the execution time decreases monotonically. This is because less SMs are being called for each kernel, so there are more available SMs for the next kernel. For this reason, a sub-sequence size of 18,320 characters was chosen for most part of the performance analysis as the default because it yields a number of blocks that comfortably fills the available H100-47 SMs without oversubscribing them (see Figure 3a).

2.4 Signature Sequence Lengths

Another factor that can significantly affect performance of the program is length of the signature. To test this, the effect of signature length on runtime was evaluated by fixing all signatures to lengths of 4,000, 6,000, 8,000, and 10,000 characters respectively. The initial hypothesis was that the runtime increases with length of the signature, since it would require longer window in order to get a valid match. However, the result shows that the trend is quite the opposite, showing negative linear slope, as shown in Figure 4

The reason could be that the number of windows for each thread to scan through is $\left\lceil \frac{l_{\text{sub}} - l_{\text{sig}}}{\text{BLOCK_SIZE}} \right\rceil$ which roughly linearly decreases with signature length, and only 1% at maximum of a sample will have any match of the substring. Hence, it is very unlikely that a thread will scan throughout the full window of size l , but rather found the mismatch and terminate the window early.

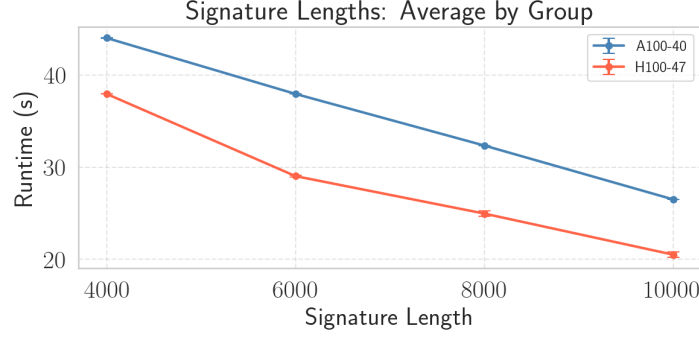


Figure 4: GPU performance based on signature length l_{sig}

3 Performance Optimization

3.1 Vectorized Memory Loading and Vectorized Shared character Matching

Copying sample data from global memory into shared memory is a high-latency operation. To reduce this overhead, `int4` (a built-in 128-bit vector type) is used to load four characters at a time. Because each shared memory bank is 4 bytes wide, loading four consecutive bytes in a single transaction produces no bank conflicts among threads in the same warp.

In addition to vectorized loads, the inner comparison loop is also vectorized. Four characters of the sample and four characters of the signature are loaded simultaneously. If all four characters match, CUDA's `__dp4a()` intrinsic is used to compute the dot product of two 4-element, 8-bit vectors in a single instruction; subtracting 132 from the result yields the sum of all four character values, which serves as the contribution to the local score. This approach reduces potential warp divergence by up to a factor of four compared with the scalar loop.

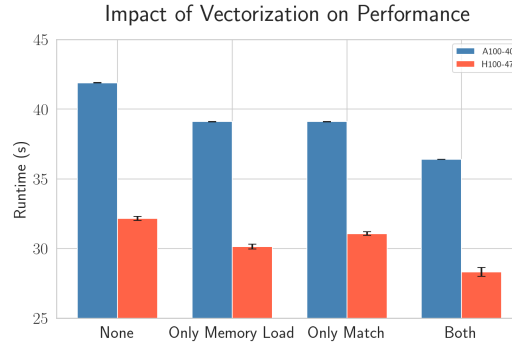


Figure 5: Runtime of Executables with and without vectorized Matching and/or vectorized character matching

As shown in Figure 5 shows that both vectorized memory load and vectorized char matching by similar amount compared to executable without any vectorized technique. Of course, combining both two technique even further reduce the execution time of the program.

Upon further investigation, we found that vectorized matching reduces warp divergence, from ~ 15 threads being active per cycle up to 23 threads per cycle. Another interesting feature is that L1 Cache miss rate increases from 10% to 30% when vectorized matching is enabled, possible due to a greater amount of addressed being load in chunk inside matching window. This might be why only-memory-load and only-match vectorization ends up having similar performance.

3.2 Concurrent Signature Matching

Rather than checking signatures sequentially, multiple signatures can be matched concurrently by exploiting the `blockDim.y` dimension of the CUDA grid. The total number of blocks is multiplied by M , and each block with index `blockIdx.y` is responsible for checking signature `blockIdx.y + i × blockDim.y` for $i = 0, 1, \dots$. However, as shown in Figure 6, runtime increases sharply with M due to SM resource contention. Because of limited number of pages, and there are already 2 optimizations done in previous part, this approach is not pursued further.

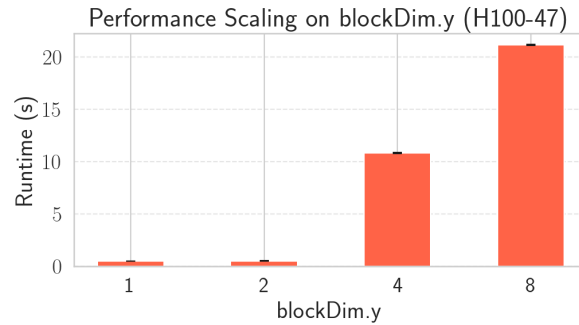


Figure 6: runtime scaling with `blockDim.y` on H100-47 performance degrades beyond $M = 2$ due to SM resource contention

4 Conclusion

The primary performance bottleneck is GPU resource saturation. When individual matching kernels already occupy the majority of available SMs, the throughput gains from CUDA streaming and concurrent signature matching are limited.

A Appendix

A.1 Raw Data

Raw data can be accessed in this [link](#).

A.2 Control Parameter used in Performance Benching

Parameters are used for performance analysis as in the following unless a parameter is being used as an independent variable, or specified otherwise:

- Number of threads per block N_{thread} : 512
- Number of signatures N_{sig} : 1000
- Sample length l_{samp} : [1000000, 2000000]
- Signature length l_{sig} : [3000, 10000]
- Number of samples without match N_{sample} : 2000
- Sample with match: 20
- Number of match per sample range: [1, 10]
- Probability of N in signature sequence: 0.1
- Probability of N in sample sequence: 0.1
- Number of Stream: 16
- *Size of subsequence per block: 18320

*18320 is being used by default because it gives optimal number of SM called per kernel call (i.e. one sample matching), see ??.

A.3 Result Repetition Guideline

A.3.1 Getting Variants of Executables

Multiple macros are used to enable automatic customized compilation. Put any of the following list after `Makefile` command to get a variant of the program:

```
SAMPLE_SUBSTRING_SIZE=<INT>      #size of sample quality and
                                   #sequence's substrings per block
GRID_MATCH=<INT>                  #blockDim.y of a grid
NUM_THREAD=<INT>                  #number of thread per block
VECTOR_MEM_LOAD=1/0              #enable/disable vectorized memory load
                                   #(1 by default)
VECTOR_MATCH=1/0                 #enable/disable vectorized character
                                   #matching (1 by default)
```

In fact, I tried to use CMake to help automate the setup, but there is some technical problem with using CUDA flag in cmake, but it didn't work out well, so it's best not to try with CMakeList.txt in repository or the performance could be worse or even show up an error (the latter case happens for me).

A.3.2 Getting Performance Result

The file `profile.sh` in repository is given. There are three main modes: normal, nsys, and ncu. Normal mode only runs the program. You can use nsys or ncu to print out Nsight system or Nsight compute files, which are helpful for analysis in more depth.

A.4 Computer Nodes used for Performance Benching

The xgpg5 node was used for A100-40 benchmarking, while an H100-47 node was used for H100-47 benchmarking.